



INFORMATIQUE

Sequence 5 : Fonctions

TD 5.1

v2025-09-24

IUT d'Annecy, 9 rue de l'Arc en Ciel, 74940 Annecy

1 TD — Introduction aux types structurés (struct)

Durée : 1h30 — Objectif

Découvrir l'intérêt des types structurés à travers un exemple concret : manipuler plusieurs données qui appartiennent à un même concept (ici, un étudiant). Les exercices montrent progressivement que la **struct** simplifie le code, le rend plus clair et plus sûr.

Exercice 1 : Le problème sans structure

On souhaite gérer les informations d'un étudiant :

- son prénom,
- son âge,
- sa moyenne.

Question 1 Proposer une liste de variables pour stocker ces informations.

Question 2 Écrire un code qui affiche les informations d'une étudiante nommée « Alice », 20 ans, moyenne 14.2.

Question 3 Comment stocker les informations de 3 étudiants différents ?

Question 4 Pourquoi cette approche devient-elle source d'erreurs ?



Les types structurés en C

En C, on peut regrouper plusieurs variables dans une seule entité appelée **structure**, définie avec le mot-clé **struct**.

Dans notre cas, nous allons créer un type structuré qui contiendra différentes informations concernant une même entité.

La définition d'un type structuré se fait généralement avec le mot-clé **typedef** pour simplifier son utilisation ultérieure. La syntaxe générale est la suivante :

```
typedef struct {  
    type1 champ1;  
    type2 champ2;  
    ...  
5   typeN champN;  
} NomDeLaStructure;
```

Prenons l'exemple d'un contact dans un carnet d'adresses. On peut définir une structure **Contact** qui regroupe plusieurs champs :

```
typedef struct {  
    char nom[50];  
    char prenom[50];  
    char telephone[15];
```

```
5 |   char email[100];  
   |   int anneeNaissance;  
   | } T_Contact;
```

Exercice 2 : Créer une structure cohérente

On regroupe les données dans une seule entité : un **type structuré**.

Question 1 A l'aide de l'exemple précédent, écris la définition d'une structure **Etudiant** contenant les champs correspondants aux informations d'un étudiant (prénom, âge, moyenne).

Question 2 Écrire les déclarations suivantes :

- une variable **etudiant_1** représentant un étudiant,
- un tableau **classe** de 30 étudiants.

Question 3 Ecrire les lignes de code permettant d'initialiser les informations de l'étudiant **etudiant_1** avec les valeurs suivantes : prénom « Bob », âge 22, moyenne 15.5.

Question 4 Ecrire les lignes de codes permettant de demander à l'utilisateur de saisir les informations des 30 étudiants du tableau **classe**.

Question 5 Ecrire les lignes de code permettant d'afficher le nom, prénom et la moyenne du meilleur étudiant de la classe.

Exercice 3 : Représenter une heure

On veut manipuler une heure sous la forme HH:MM:SS.

Question 1 Proposer un type structuré **T_Heure** permettant de stocker les informations d'un horaire.

Question 2 Écrire une fonction void **afficher_heure(T_Heure t)** qui affiche HH:MM:SS.

Question 3 Écrire int **en_secondes(T_Heure)** qui renvoie le nombre total de secondes.

Question 4 Créer deux horaires dans le **main** et afficher la différence en secondes.

Exercice 4 : Gérer un produit en stock

On veut garder les informations suivantes pour chaque produit :

- son nom (chaîne de caractères),
- son prix HT (nombre à virgule),
- son taux de TVA (nombre à virgule).

Question 1 Proposer un type structuré **T_Produit** pour stocker ces informations.

Question 2 Écrire une fonction qui renvoie le prix TTC d'un produit (la fonction ne prendra qu'un seul argument.).

Exercice 5 : Gestion d'un vecteur 2D

On manipule des vecteurs dans un plan :

```
typedef struct{  
    float x;  
    float y;  
} T_Vecteur;
```

Question 1 Écrire une fonction qui renvoie la norme $\sqrt{x^2 + y^2}$ d'un vecteur donné en paramètre.

Question 2 Écrire une fonction permettant de sommer deux vecteurs.

Question 3 Écrire une fonction permettant de donner l'opposé d'un vecteur.

Question 4 Dans le **main**, déclarer trois vecteurs, afficher leurs normes, puis afficher la norme de $a - b$.

Exercice 6 : Extension : struct imbriquées

On veut maintenant stocker les coordonnées d'un étudiant dans une structure **Adresse**.

```
struct Adresse {  
    char rue[50];  
    int codePostal;  
    char ville[30];  
};  
  
struct Etudiant {  
    char prenom[30];  
    int age;  
    float moyenne;  
    struct Adresse adresse;  
};
```

Question 1 Comment accéder à la ville d'un étudiant ?

Question 2 Quel est l'avantage de créer une structure imbriquée plutôt qu'ajouter 3 champs à **Etudiant** ?